

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

REPUBLIC OF CAMEROON

Peace – Work – Fatherland

Universite de Yaounde I

University of Yaounde I



Ecole Nationale Supérieure Polytechnique de Yaounde

National Advanced School of Engineering of Yaounde

Departement du Genie Informatique

UNITE D'ENSEIGNEMENT

Intelligence Artificielle et Applications

Module : ANI-IA 4

RAPPORT DE PROJET

Projet 1 — Systeme de Recommandation

Reseaux de Neurones Convolutifs sur Graphes (LightGCN)

Presente par :

Nom : BILOA ABADJECK

Prenom : Valery Paolo Stelane

Matricule : 22P009

Classe : ANI-IA 4

Encadre par :

M. OMGBA BITHA Junior

Departement du Genie Informatique

ENSPY Yaounde

Annee academique 2025/2026

Resume

Ce rapport presente la realisation complete du **Projet 1** du module *Intelligence Artificielle et Applications (ANI-IA 4)* portant sur la conception et l'implementation d'un **systeme de recommandation de films** base sur le modele **LightGCN** (Light Graph Convolutional Network).

LightGCN est une architecture de reseau de neurones sur graphes, proposee par He et al. (2020), specialement concue pour les systemes de recommandation collaboratifs. Le modele exploite les interactions utilisateur-article sous forme d'un **graphe biparti** et propage les embeddings a travers K couches de convolution, sans transformation non lineaire, ce qui le rend particulierement efficace et leger.

Le projet a ete realise entierement sur **Visual Studio Code** sous Windows avec Python 3.14, en utilisant PyTorch comme framework principal. Le dataset utilise est **MovieLens ml-latest-small**, contenant 100 836 evaluations de films par 610 utilisateurs sur 9 742 films.

Resultats obtenus :

- Recall@10 = **0.0787** (meilleur a l'epoque 90)
- NDCG@10 = **0.0323**
- Convergence de la perte BPR de 0.677 a 0.150 sur 100 epoques
- Interface de visualisation en temps reel avec Flask et Socket.IO

Mots-cles : LightGCN, Systeme de Recommandation, Graphe Biparti, BPR Loss, Recall@K, NDCG@K, PyTorch, MovieLens, Transfer Learning.

Contents

Resume	1
1 Introduction	5
1.1 Contexte	5
1.2 Objectifs du Projet	5
1.3 Organisation du Rapport	6
2 Etat de l'Art	7
2.1 Systemes de Recommandation	7
2.2 Factorisation de Matrices	7
2.3 Reseaux de Neurons sur Graphes (GCN)	7
2.4 LightGCN (He et al., 2020)	8
2.5 Perte BPR (Bayesian Personalized Ranking)	8
3 Methodologie	9
3.1 Arborescence du Projet	9
3.2 Environnement de Developpement	10
3.3 Dataset MovieLens	10
3.4 Pipeline du Projet	10
3.5 Hyperparametres du Modele	11
4 Implementation	12
4.1 Fichier src/config.py — Configuration Centrale	12
4.2 Fichier src/model.py — Architecture LightGCN	13
4.3 Fichier src/graph.py — Construction du Graphe	14
4.4 Fichier src/evaluator.py — Metriques	15
5 Mode d'Emploi	17
5.1 Prerequis	17
5.2 Installation pas a pas	17
5.2.1 Etape 1 : Creer le dossier et l'environnement virtuel	17
5.2.2 Etape 2 : Installer les dependances	18
5.2.3 Etape 3 : Creer la structure de dossiers	18

5.2.4	Etape 4 : Telecharger le dataset MovieLens	18
5.2.5	Etape 5 : Copier tous les fichiers source	19
5.3	Lancement du Pipeline Principal	19
5.4	Lancement du Dashboard Temps Reel	19
6	Resultats	21
6.1	Resultats de Performance	21
6.2	Evolution des Metriques par Epoque	21
6.3	Exemples de Recommandations Generees	22
6.4	Interface du Dashboard	22
7	Difficultes Rencontrees et Solutions	23
7.1	Python 3.14 incompatible avec PyTorch GPU	23
7.2	UserWarning : Sparse Invariant Checks	23
7.3	n_iter deprecie dans scikit-learn TSNE	23
7.4	Fichier main.py cree dans le mauvais dossier	24
7.5	Temps d'entrainement long sur CPU	24
8	Conclusion et Perspectives	25
8.1	Bilan du Projet	25
8.2	Limites et Perspectives	25
8.3	Reflexion Finale	26
	Remerciements	27
	References Bibliographiques	28
	Glossaire	29

List of Figures

List of Tables

3.1	Environnement de developpement utilise	10
3.2	Caracteristiques du dataset MovieLens ml-latest-small	10
3.3	Hyperparametres de LightGCN	11
6.1	Evolution des metriques au cours de l'entrainement	21
6.2	Recommandations Top-10 pour 3 utilisateurs selectionnes	22

Chapter 1

Introduction

1.1 Contexte

Les systemes de recommandation constituent aujourd’hui un element fondamental des plateformes numeriques modernes. Que ce soit Netflix pour les films, Spotify pour la musique, Amazon pour les produits ou YouTube pour les videos, ces systemes permettent de proposer du contenu pertinent a chaque utilisateur en exploitant l’historique de ses interactions passees.

Les approches classiques de *filtrage collaboratif* par factorisation de matrices presentent des limites importantes face a des donnees eparses et des relations complexes entre utilisateurs et articles. C’est dans ce contexte que **LightGCN** (He et al., 2020) a ete propose, en modelisant le graphe d’interaction utilisateur-article a travers un reseau de neurones convolutifs simplifie, sans transformation de caracteristiques ni activation non lineaire.

Ce projet s’inscrit dans le cadre du module *Intelligence Artificielle et Applications (ANI-IA 4)* a l’Ecole Nationale Superieure Polytechnique de Yaounde (ENSPY), sous l’encadrement de M. OMGBA BITHA Junior.

1.2 Objectifs du Projet

Les objectifs principaux de ce projet sont :

- Comprendre les fondements theoriques des reseaux de neurones convolutifs sur graphes (GCN) et leur simplification via LightGCN
- Construire un graphe biparti representant les interactions utilisateur-article a partir du dataset MovieLens
- Implementer LightGCN en PyTorch avec propagation d’embeddings multicouches et perte BPR

- Evaluer les performances avec les metriques Recall@K et NDCG@K
- Generer des recommandations Top-K personnalisées
- Developper un tableau de bord interactif de visualisation en temps reel

1.3 Organisation du Rapport

Ce rapport s'articule autour des chapitres suivants :

1. **Introduction** : contexte, objectifs et organisation
2. **Etat de l'Art** : revue des travaux existants en systemes de recommandation et GCN
3. **Methodologie** : arborescence, environnement et pipeline complet
4. **Implementation** : codes sources commentes fichier par fichier
5. **Mode d'Emploi** : installation et lancement pas a pas
6. **Resultats** : metriques, graphiques et captures d'ecran
7. **Difficultes Rencontrees** : problemes et solutions apportees
8. **Conclusion et Perspectives**

Chapter 2

Etat de l'Art

2.1 Systemes de Recommandation

Les systemes de recommandation se divisent en trois grandes categories :

Filtrage base sur le contenu : Recommande des articles similaires a ceux qu'un utilisateur a deja apprecies, en se basant sur les caracteristiques des articles (genre, realisateur, acteurs pour les films).

Filtrage collaboratif : Exploite les comportements collectifs des utilisateurs. Si deux utilisateurs ont des gouts similaires, le systeme recommande a l'un ce que l'autre a apprecie. C'est l'approche la plus utilisee en pratique.

Approches hybrides : Combinent les deux precedentes pour pallier leurs limitations respectives.

2.2 Factorisation de Matrices

La factorisation de matrices (Matrix Factorization) a ete la methode dominante en filtrage collaboratif pendant de nombreuses annees. Le principe est de decomposer la matrice d'interactions utilisateur-article R en deux matrices de faible rang :

$$R \approx P \cdot Q^T \tag{2.1}$$

ou $P \in \mathbb{R}^{M \times d}$ contient les embeddings des M utilisateurs et $Q \in \mathbb{R}^{N \times d}$ ceux des N articles, d etant la dimension des embeddings. Cette approche presente des limitations face aux donnees tres eparses et ne capture pas les relations d'ordre superieur.

2.3 Reseaux de Neurones sur Graphes (GCN)

Les reseaux de neurones sur graphes (Graph Convolutional Networks, GCN) permettent d'apprendre des representations de noeuds dans un graphe en agregant les informations des

voisins. Pour un graphe d'interaction utilisateur-article, les GCN capturent les relations de collaboration de maniere implicite : si deux utilisateurs ont interagi avec les memes articles, leurs representations se rapprochent dans l'espace d'embedding.

2.4 LightGCN (He et al., 2020)

LightGCN est une simplification radicale des GCN pour la recommandation. Les auteurs ont montre que les deux composantes les plus coutteuses des GCN — la transformation lineaire et l'activation non lineaire — sont inutiles, voire nuisibles, pour la recommandation collaborative.

La formule de propagation de LightGCN a la couche k est :

$$E^{(k)} = \hat{A} \cdot E^{(k-1)} \quad (2.2)$$

ou $\hat{A} = D^{-1/2} \tilde{A} D^{-1/2}$ est la matrice d'adjacence normalisee du graphe biparti, et $E^{(k)}$ est la matrice d'embeddings a la couche k .

L'embedding final est obtenu par moyenne de toutes les couches :

$$E^* = \frac{1}{K+1} \sum_{k=0}^K E^{(k)} \quad (2.3)$$

Le score de prediction pour une paire (utilisateur u , article i) est :

$$\hat{y}_{ui} = e_u^{*\top} \cdot e_i^* \quad (2.4)$$

2.5 Perte BPR (Bayesian Personalized Ranking)

La perte BPR (Rendle et al., 2009) est specifiquement concue pour l'apprentissage de classements personnalisés. Elle optimise le fait que l'utilisateur prefere un article interagi (positif) a un article non interagi (negatif) :

$$\mathcal{L}_{BPR} = - \sum_{(u,i,j)} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|E^{(0)}\|^2 \quad (2.5)$$

ou j est un article negatif tire aleatoirement, et λ est le coefficient de regularisation L2.

Chapter 3

Methodologie

3.1 Arborescence du Projet

La structure complete du projet est la suivante :

Arborescence — projet1_lightgcn/

```
projet1_lightgcn/
|
+-- .venv/                # Environnement virtuel Python
+-- data/
|   +-- ml-latest-small/  # Dataset MovieLens
|       +-- ratings.csv   # 100 836 evaluations
|       +-- movies.csv    # 9 742 films
|       +-- tags.csv
|       +-- links.csv
|
+-- src/
|   +-- __init__.py       # Package Python
|   +-- config.py         # Hyperparametres centraux
|   +-- dataset.py        # Chargement et preprocessing
|   +-- graph.py          # Construction graphe biparti
|   +-- model.py          # Architecture LightGCN
|   +-- trainer.py        # Boucle d'entrainement BPR
|   +-- evaluator.py      # Metriques Recall@K, NDCG@K
|   +-- utils.py          # Visualisation et utilitaires
|
+-- outputs/
|   +-- models/           # Modeles sauvegardes (.pth)
|   +-- figures/          # Graphiques generes (.png)
|   +-- results/          # Fichiers CSV de resultats
|
```

```

+-- main.py           # Point d'entree principal
+-- dashboard.py      # Tableau de bord temps reel
+-- dashboard.html    # Interface web du dashboard
+-- requirements.txt   # Dependances Python

```

3.2 Environnement de Developpement

Table 3.1: Environnement de developpement utilise

Composant	Version / Detail
Systeme d'exploitation	Windows 11
IDE	Visual Studio Code
Terminal	PowerShell
Python	3.14.5
PyTorch	2.12.0+cpu
NumPy	2.4.4
Pandas	2.x
Scikit-learn	1.3+
Matplotlib/Seaborn	3.7+
Flask	3.x
Flask-SocketIO	5.x
GPU	Non utilise (CPU uniquement)

3.3 Dataset MovieLens

Le dataset utilise est **MovieLens ml-latest-small**, disponible gratuitement sur le site GroupLens (<https://grouplens.org/datasets/movielens/>).

Table 3.2: Caracteristiques du dataset MovieLens ml-latest-small

Caracteristique	Valeur
Nombre d'evaluations brutes	100 836
Nombre d'utilisateurs (apres filtrage)	610
Nombre d'articles (apres filtrage)	3 650
Interactions d'entrainement	89 664
Interactions de test	610
Strategie de split	Leave-one-out
Filtrage minimum	5 interactions par user/item

3.4 Pipeline du Projet

Le pipeline complet se deroule en 5 phases sequentielles :

1. **Chargement des donnees** : lecture de `ratings.csv`, filtrage des utilisateurs et articles avec moins de 5 interactions, reindexation des identifiants en indices continus 0..N-1
2. **Construction du graphe biparti** : creation de la matrice d'interaction R , construction de $\tilde{A}$, normalisation symetrique $\hat{A} = D^{-1/2}\tilde{A}D^{-1/2}$
3. **Initialisation du modele** : embeddings initiaux $E^{(0)}$ pour utilisateurs et articles (dimension 64, initialisation normale)
4. **Entrainement** : 100 epoques avec Adam (lr=0.001), echantillonnage negatif, perte BPR, evaluation tous les 10 epoques
5. **Evaluation et visualisation** : Recall@10, NDCG@10, t-SNE des embeddings, recommandations Top-10

3.5 Hyperparametres du Modele

Table 3.3: Hyperparametres de LightGCN

Hyperparametre	Valeur	Role
Dimension embeddings	64	Taille des vecteurs E
Couches de propagation K	3	Profondeur de propagation
Taux d'apprentissage	0.001	Pas de gradient Adam
Regularisation L2	10^{-4}	Penalisation des poids
Taille des batches	1024	Echantillons par iteration
Nombre d'epoques	100	Iterations completes
Evaluation tous les	10 epoques	Frequence d'evaluation
Top-K	10	Taille de la liste de reco
Graine aleatoire	42	Reproductibilite

Chapter 4

Implementation

4.1 Fichier src/config.py — Configuration Centrale

Tous les hyperparametres sont centralises dans ce fichier. Modifier ici suffit pour changer un parametre dans tout le projet.

```
1 import os
2
3 BASE_DIR    = os.path.dirname(os.path.dirname(os.path.abspath(
4     __file__)))
5 DATA_DIR   = os.path.join(BASE_DIR, "data", "ml-latest-small")
6 OUTPUT_DIR  = os.path.join(BASE_DIR, "outputs")
7 MODELS_DIR  = os.path.join(OUTPUT_DIR, "models")
8 FIGURES_DIR = os.path.join(OUTPUT_DIR, "figures")
9 RESULTS_DIR = os.path.join(OUTPUT_DIR, "results")
10
11 RATINGS_FILE = os.path.join(DATA_DIR, "ratings.csv")
12 MOVIES_FILE  = os.path.join(DATA_DIR, "movies.csv")
13
14 # Filtrage des donnees
15 MIN_INTERACTIONS = 5
16 RANDOM_SEED      = 42
17
18 # Modele LightGCN
19 EMBEDDING_DIM = 64      # Dimension des embeddings
20 NUM_LAYERS    = 3       # Nombre de couches K
21
22 # Entraînement
23 LEARNING_RATE = 0.001
24 WEIGHT_DECAY  = 1e-4
25 BATCH_SIZE    = 1024
26 NUM_EPOCHS    = 100
```

```

26 EVAL EVERY      = 10
27 SEED            = 42
28
29 # Evaluation
30 TOP_K = 10

```

Listing 4.1: src/config.py — Configuration centrale

4.2 Fichier src/model.py — Architecture LightGCN

```

1 import torch
2 import torch.nn as nn
3 from src.config import EMBEDDING_DIM, NUM_LAYERS, WEIGHT_DECAY
4
5 class LightGCN(nn.Module):
6     def __init__(self, n_users, n_items,
7                   embedding_dim=EMBEDDING_DIM,
8                   n_layers=NUM_LAYERS):
9         super(LightGCN, self).__init__()
10        self.n_users      = n_users
11        self.n_items      = n_items
12        self.embedding_dim = embedding_dim
13        self.n_layers     = n_layers
14
15        # Embeddings initiaux -- SEULS parametres apprenables
16        self.user_embedding = nn.Embedding(n_users, embedding_dim)
17        self.item_embedding = nn.Embedding(n_items, embedding_dim)
18
19        nn.init.normal_(self.user_embedding.weight, std=0.1)
20        nn.init.normal_(self.item_embedding.weight, std=0.1)
21
22        def forward(self, adj_matrix):
23            # Concatener users et items : (n_users+n_items, dim)
24            E = torch.cat([self.user_embedding.weight,
25                           self.item_embedding.weight], dim=0)
26            all_layers = [E]
27
28            # K couches de propagation :  $E^{(k)} = \hat{A} * E^{(k-1)}$ 
29            for _ in range(self.n_layers):
30                E = torch.sparse.mm(adj_matrix, E)
31                all_layers.append(E)

```

```

31
32     # Agregation par moyenne :  $E^* = \text{mean}(E^0, \dots, E^K)$ 
33     E_final = torch.stack(all_layers, dim=1).mean(dim=1)
34     return E_final[:self.n_users], E_final[self.n_users:]
35
36 def bpr_loss(self, adj_matrix, users, pos_items, neg_items):
37     user_emb, item_emb = self.forward(adj_matrix)
38     u = user_emb[users]
39     pos = item_emb[pos_items]
40     neg = item_emb[neg_items]
41     pos_scores = (u * pos).sum(dim=1)
42     neg_scores = (u * neg).sum(dim=1)
43     bpr = -torch.log(
44         torch.sigmoid(pos_scores - neg_scores) + 1e-8).mean()
45     reg = WEIGHT_DECAY * (
46         self.user_embedding.weight[users].norm(2).pow(2) +
47         self.item_embedding.weight[pos_items].norm(2).pow(2)
48 +
49         self.item_embedding.weight[neg_items].norm(2).pow(2)
50     ) / len(users)
51     return bpr + reg

```

Listing 4.2: src/model.py — Architecture LightGCN (extrait)

4.3 Fichier src/graph.py — Construction du Graphe

```

1 import numpy as np
2 import torch
3 import scipy.sparse as sp
4
5 def build_adjacency_matrix(train_data, n_users, n_items):
6     """Construit  $\hat{A} = D^{-1/2} * \tilde{A} * D^{-1/2}$ """
7     rows, cols = [], []
8     for user, items in train_data.items():
9         for item in items:
10             rows.append(user)
11             cols.append(item)
12
13     R = sp.csr_matrix(
14         (np.ones(len(rows)), (rows, cols)),
15         shape=(n_users, n_items))
16

```



```

17     # Graphe biparti A_tilde
18     top      = sp.hstack([sp.csr_matrix((n_users,n_users)), R])
19     bottom   = sp.hstack([R.T, sp.csr_matrix((n_items,n_items))])
20     A_tilde  = sp.vstack([top, bottom]).tocsr()
21
22     # Normalisation symetrique
23     rowsum    = np.array(A_tilde.sum(axis=1)).flatten()
24     d_inv_sqrt = np.power(rowsum + 1e-10, -0.5)
25     D_inv_sqrt = sp.diags(d_inv_sqrt)
26     A_hat     = D_inv_sqrt.dot(A_tilde).dot(D_inv_sqrt).tocoo()
27
28     # Conversion torch.sparse
29     indices   = torch.LongTensor(np.vstack([A_hat.row, A_hat.col]))
30     values    = torch.FloatTensor(A_hat.data)
31     return torch.sparse_coo_tensor(indices, values, A_hat.shape)

```

Listing 4.3: src/graph.py — Construction de la matrice normalisee

4.4 Fichier src/evaluator.py — Metriques

```

1  import torch, numpy as np
2  from src.config import TOP_K
3
4  class Evaluator:
5      def __init__(self, top_k=TOP_K):
6          self.top_k = top_k
7
8      def evaluate(self, model, adj_matrix, train_data,
9                  test_data, device):
10         model.eval()
11         recalls, ndcgs = [], []
12         with torch.no_grad():
13             user_emb, item_emb = model.forward(adj_matrix)
14             for user_idx, test_items in test_data.items():
15                 if not test_items: continue
16                 scores = torch.matmul(user_emb[user_idx],
17                                     item_emb.t())
18                 scores[train_data.get(user_idx, [])] = -float('inf')
19             _, top_k_idx = torch.topk(scores, self.top_k)
20             top_k = top_k_idx.cpu().numpy().tolist()
21

```

```
22         # Recall@K
23         hits = len(set(top_k) & set(test_items))
24         recall = hits / min(len(test_items), self.top_k)
25         recalls.append(recall)
26
27         # NDCG@K
28         dcg = sum(1/np.log2(i+2) for i,x in
29                 enumerate(top_k) if x in set(test_items
30             ))
31         idcg = sum(1/np.log2(i+2) for i in
32                 range(min(len(test_items),self.top_k))
33             )
34         ndcgs.append(dcg/idcg if idcg else 0.0)
35
36     return float(np.mean(recalls)), float(np.mean(ndcgs))
```

Listing 4.4: src/evaluator.py — Recall@K et NDCG@K

Chapter 5

Mode d'Emploi

5.1 Prerequis

Avant de demarrer, s'assurer d'avoir :

- Python 3.10 ou superieur installe
- Visual Studio Code installe
- Connexion internet pour telecharger MovieLens
- Au moins 2 Go d'espace disque disponible

5.2 Installation pas a pas

5.2.1 Etape 1 : Creer le dossier et l'environnement virtuel

PowerShell — Creation de l'environnement

```
# Naviguer vers le bureau
cd C:\Users\PAOLO\Desktop

# Creer le dossier projet
mkdir projet1_lightgcn
cd projet1_lightgcn

# Creer l'environnement virtuel
python -m venv .venv

# Activer l'environnement
.venv\Scripts\Activate.ps1

# Signe que c'est active : (.venv) apparait
```

5.2.2 Etape 2 : Installer les dependances

PowerShell — Installation des bibliotheques

```
# Installer PyTorch (version CPU)
pip install torch torchvision --index-url ^
    https://download.pytorch.org/whl/cpu

# Installer les autres dependances
pip install numpy pandas scikit-learn scipy ^
    matplotlib seaborn tqdm jupyter ipykernel

# Installer Flask pour le dashboard
pip install flask flask-socketio
```

5.2.3 Etape 3 : Creer la structure de dossiers

PowerShell — Creation de la structure

```
# Creer tous les dossiers necessaires
mkdir src, data, outputs
mkdir outputs\models, outputs\figures, outputs\results

# Creer le fichier __init__.py
New-Item src\__init__.py -ItemType File
```

5.2.4 Etape 4 : Telecharger le dataset MovieLens

1. Aller sur <https://grouplens.org/datasets/movielens/latest/>
2. Telecharger ml-latest-small.zip
3. Extraire le contenu dans le dossier data/
4. Verifier la presence de ratings.csv

PowerShell — Verification du dataset

```
# Verifier que le dataset est en place
ls data\ml-latest-small\

# Resultat attendu :
# links.csv      movies.csv      ratings.csv     tags.csv
```

5.2.5 Etape 5 : Copier tous les fichiers source

Copier les fichiers `config.py`, `dataset.py`, `graph.py`, `model.py`, `trainer.py`, `evaluator.py`, `utils.py` dans le dossier `src/`, puis `main.py`, `dashboard.py` et `dashboard.html` a la racine du projet.

5.3 Lancement du Pipeline Principal

PowerShell — Lancement de main.py

```
# S'assurer que l'environnement est active
.venv\Scripts\Activate.ps1

# Lancer le pipeline complet
python main.py

# Sortie attendue :
# [Dataset] 610 utilisateurs, 3650 articles
# [Graph] Matrice A_hat : 4260x4260
# [LightGCN] params=272,640
# Ep 1/100 | Loss=0.6776 | Recall@10=0.0443
# ...
# Ep 100/100 | Loss=0.1495 | Recall@10=0.0770
# Recall@10 : 0.0787
# NDCG@10 : 0.0323
```

Duree estimee : 15 a 25 minutes sur CPU.

5.4 Lancement du Dashboard Temps Reel

PowerShell — Lancement du dashboard

```
# Lancer le serveur Flask
python dashboard.py

# Sortie attendue :
# [Dashboard] Ouvre http://localhost:5000
# * Running on http://127.0.0.1:5000

# Ouvrir dans le navigateur :
# http://localhost:5000

# Cliquer sur "Lancer LightGCN"
```

Observer l'entraînement en temps réel

Chapter 6

Resultats

6.1 Resultats de Performance

Apres 100 epoques d'entrainement sur le dataset MovieLens ml-latest-small, le modele LightGCN atteint les performances suivantes :

Resultats finaux — LightGCN sur MovieLens	
Recall@10 (meilleur)	0.0787 (epoque 90)
NDCG@10 (meilleur)	0.0323 (epoque 90)
Loss BPR initiale	0.6776 (epoque 1)
Loss BPR finale	0.1495 (epoque 100)
Reduction de la perte	-77.9%
Amelioration Recall	+77.7% (0.0443 -> 0.0787)

6.2 Evolution des Metriques par Epoque

Table 6.1: Evolution des metriques au cours de l'entrainement

Epoque	Loss BPR	Recall@10	NDCG@10
1	0.6776	0.0443	0.0239
10	0.3458	0.0508	0.0271
20	0.3157	0.0508	0.0256
30	0.2829	0.0475	0.0249
40	0.2470	0.0508	0.0249
50	0.2247	0.0607	0.0264
60	0.1966	0.0623	0.0276
70	0.1816	0.0672	0.0285
80	0.1702	0.0721	0.0306
90	0.1599	0.0787	0.0323
100	0.1495	0.0770	0.0309

6.3 Exemples de Recommandations Generees

Table 6.2: Recommandations Top-10 pour 3 utilisateurs selectionnes

User	Rang	Film recommande
(215 films vus)	1	Twister (1996)
	2	American Pie (1999)
	3	Armageddon (1998)
(517 films vus)	1	Forrest Gump (1994)
	2	Alien (1979)
	3	Braveheart (1995)
(155 films vus)	1	The Fugitive (1993)
	2	Star Wars: Episode VI (1983)
	3	Pulp Fiction (1994)

6.4 Interface du Dashboard

Le tableau de bord interactif developpe avec Flask et Socket.IO permet de surveiller en temps reel l'execution du pipeline. Il comprend :

- 4 cartes de phases avec barres de progression animees
- Barre de progression globale epoque par epoque
- Affichage en direct de la Loss, Recall@10 et NDCG@10
- Deux graphiques Chart.js (courbe de perte + metriques)
- Section des recommandations Top-10 avec onglets par utilisateur
- Conclusion automatique avec analyse des resultats

Chapter 7

Difficultes Rencontrees et Solutions

7.1 Python 3.14 incompatible avec PyTorch GPU

Probleme : PyTorch ne supporte pas encore Python 3.14 avec CUDA. La commande suivante echouait :

Erreur rencontree

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
# Erreur : Could not find a version that satisfies the requirement torch
```

Solution : Utilisation de la version CPU de PyTorch, entierement fonctionnelle pour ce projet. Le modele LightGCN sur MovieLens small s'entraine en 15-25 minutes sur CPU, ce qui est acceptable.

Solution appliquee

```
pip install torch torchvision ~
--index-url https://download.pytorch.org/whl/cpu
```

7.2 UserWarning : Sparse Invariant Checks

Probleme : Warning affiche lors de la creation de la matrice sparse :

UserWarning: Sparse invariant checks are implicitly disabled.

Impact : Aucun. C'est un avertissement de performance de PyTorch, pas une erreur. Le calcul est correct.

Solution : Ignorer ce warning. Il n'affecte pas les resultats.

7.3 n_iter deprecie dans scikit-learn TSNE

Probleme : Erreur lors de la visualisation t-SNE :

`TypeError: TSNE.__init__() got an unexpected keyword argument 'n_iter'`

Solution : Remplacer `n_iter` par `max_iter` dans scikit-learn 1.5+ :

Correction appliquée

```
# Avant (erreur)
tsne = TSNE(n_components=2, n_iter=1000)

# Apres (correct)
tsne = TSNE(n_components=2, max_iter=1000)
```

7.4 Fichier `main.py` cree dans le mauvais dossier

Probleme : `main.py` avait ete cree dans `src/` au lieu de la racine du projet.

Solution : Utiliser PowerShell pour le deplacer :

Correction du chemin

```
Move-Item src\main.py main.py
```

7.5 Temps d'entrainement long sur CPU

Probleme : L'entrainement de 100 epoques prend 15-25 minutes sur CPU, ce qui est penalisant lors des tests.

Solution : Reduire `NUM_EPOCHS` a 20 dans `config.py` pour les tests rapides, puis remettre a 100 pour l'execution finale.

Chapter 8

Conclusion et Perspectives

8.1 Bilan du Projet

Ce projet a permis de realiser une implementation complete et fonctionnelle de LightGCN pour la recommandation de films. Le modele atteint un **Recall@10 de 0.0787** et un **NDCG@10 de 0.0323** apres 100 epoques d'entrainement sur MovieLens ml-latest-small, ce qui constitue un resultat satisfaisant pour cette configuration (dataset reduit, 610 utilisateurs, CPU uniquement).

Les points forts du projet sont :

- Implementation from scratch de LightGCN sans librairies de haut niveau
- Pipeline complet de A a Z : preprocessing, graphe, modele, entrainement, evaluation
- Dashboard interactif en temps reel avec Flask et Socket.IO
- Code modulaire et bien structure

8.2 Limites et Perspectives

Limites identifiees :

- Dataset reduit (610 users) : les performances seraient meilleures sur MovieLens-1M
- Absence de GPU : l'entrainement est lent sur CPU
- Strategie leave-one-out severe : 1 seul film de test par user

Perspectives d'amelioration :

- Passer a MovieLens-1M (6000 users, 4000 films) pour des metriques plus representatives
- Augmenter la dimension des embeddings a 128 et K a 4 couches

- Entraîner 200+ époques avec GPU pour de meilleures performances
- Intégrer des features de contenu (genres, années) pour une approche hybride
- Déployer en production avec une API REST

8.3 Reflexion Finale

Ce projet illustre parfaitement les défis pratiques du deep learning appliqué aux systèmes de recommandation : la qualité des données, le choix de l'architecture, et la disponibilité des ressources de calcul sont des facteurs déterminants. LightGCN, malgré sa simplicité apparente, capture efficacement les structures de collaboration implicite dans les graphes d'interaction.

Remerciements

Je tiens à exprimer ma profonde gratitude à :

- **M. OMGBA BITHA Junior**, pour son encadrement, ses conseils et sa disponibilité tout au long de ce projet
- **L'Ecole Nationale Supérieure Polytechnique de Yaounde (ENSPY)**, pour les ressources mises à disposition
- **La communauté open-source**, pour les outils utilisés : PyTorch, scikit-learn, Flask, Chart.js
- **GroupLens Research**, pour le dataset MovieLens disponible gratuitement

References Bibliographiques

1. **He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M.** (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*. ACM.
2. **Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L.** (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI 2009)*.
3. **Harper, F. M., & Konstan, J. A.** (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4), 1-19.
4. **Paszke, A., et al.** (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.
5. **Koren, Y., Bell, R., & Volinsky, C.** (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8), 30-37.

Glossaire

BPR	Bayesian Personalized Ranking. Fonction de perte pour l'apprentissage de classements personnalisés dans les systèmes de recommandation.
Embedding	Vecteur de nombres réels représentant un utilisateur ou un article dans un espace de basse dimension. Capture les préférences et caractéristiques de manière dense.
FAISS	Facebook AI Similarity Search. Bibliothèque de recherche par similarité vectorielle optimisée.
Graphe biparti	Graphe dont les nœuds se divisent en deux ensembles disjoints (utilisateurs et articles), sans arêtes entre nœuds du même ensemble.
GCN	Graph Convolutional Network. Architecture de réseau de neurones permettant d'apprendre des représentations de nœuds dans un graphe.
Leave-one-out	Stratégie de split : pour chaque utilisateur, la dernière interaction est réservée pour le test, le reste pour l'entraînement.
LightGCN	Light Graph Convolutional Network. Simplification des GCN pour la recommandation : pas de transformation linéaire ni d'activation non linéaire.
NDCG@K	Normalized Discounted Cumulative Gain at K. Métrique de qualité du classement : récompense les items pertinents placés en haut de la liste de recommandations.
Recall@K	Proportion d'items pertinents retrouvés dans le Top-K recommandé. Mesure la couverture des recommandations.
Socket.IO	Bibliothèque permettant la communication bidirectionnelle en temps réel entre un serveur Flask et un navigateur web.
t-SNE	t-Distributed Stochastic Neighbor Embedding. Technique de visualisation de données de haute dimension en 2D ou 3D.

Top-K Les K meilleurs items recommandes a un utilisateur, classes par score de similarite decroissant.